

Анализ статьи "Рефакторинг — это как весенняя уборка: никто не хочет её начинать, но все ценит чистоту и порядок после её завершения" (автор анализа — Дмитрий Придионов, 1 курс, 12-25 Июн, Программная инженерия).

Главная мысль статьи: Рефакторинг — это не роскошь, а гигиена. Если его игнорировать, проект превращается в свалку «костылей», где даже простая смена константы требует написания функции с `if else`.

Автор приводит шикарные примеры деградации кода. Вот что происходит, когда на чистоту забывают: 1) Размытие абстракций: Константы, которые внезапно становятся логическими функциями. Это нарушает принцип единственной ответственности. 2) Потеря типизации: Когда типы превращаются в объекты с раздатными значениями «на коленке». 3) Зависимости: Переде (!) просто на обновление библиотек. Это прямой результат накопленного технического долга.

В статье четко подмечено, как формат работы влияет на качество:

- * Agile (Scrum) —> Итеративно, каждый спринт. —> Идеальный мир. Код всегда «свежий».
- * Waterfall —> Отдельными этапами в конце вехи. —> Опасно. До этого этапа можно не дожить или погрязнуть в багах.



Troonsply

22 мая 2024 в 16:14

Рефакторинг — это как весенняя уборка: никто не хочет её начинать, но все ценят чистоту и порядок после её завершения



Простой



4 мин



1.4K

Веб-разработка*, Программирование*, Качество кода*, Говнокод, Проектирование и рефакторинг*

[Мнение](#)

Я работаю в аутсорсе и недавно вернулась на один из проектов, с которого ушла год назад. К своему глубокому удивлению, ознакомившись с кодом, я поняла, что его никто не рефакторил весь этот год. При разговоре с коллегой, который работал над этим проектом, я выяснила, что время на рефакторинг отдельно не выделялось, и он его не успел провести.

Итак, что я имею сейчас? Что стало с кодом, который не рефакторили год? Вопрос риторический, и так понятно, что он превратился в легаси.

Например, эти замечательные константы под гнетом измененных требований превратились в функции:

Было:

```
const CONST_GET_TEST = "CONST_GET_TEST";
```

[Объяснить с](#) SourceCraft**Стало:**

```
const CONST_GET_TEST = (isTest: boolean) => {
  if (isTest) {
    return "CONST_GET_TEST_1";
  } else {
    return "CONST_GET_TEST_2";
  }
};
```

[Объяснить с](#)

Или вот эти замечательные объекты, которые по-видимому раньше были типами, внезапно превратились в переменные с initial state:

Было:

```
const Person = { id: number, name: string, age: number };
const Info = { id: number, desc: string, old: number };
const Level = { id: number, level: number };
```

[Объяснить с](#) **Стало:**

```
const Person = { id: 0, name: "", age: 0 };
const Info = { id: 0, desc: "", old: 0 };
const Level = { id: 0, level: 0 };
```

Объяснить с 

Этот код получил право на существование только потому, что разработчику никто не выделил времени на рефакторинг, и ему пришлось словно “на коленке” дописывать код (надеюсь, что это всё-таки не какое-то предвзятое отношение ко мне).

На обновление одной из библиотек я потратила неделю, справляясь со всеми болями несовместимости с другими либрами в проекте, очень хотелось оставить как есть, но устаревшие версии библиотек стали бы проблемами попозже, они никуда бы не делись. Примерно три месяца ушло на то, чтобы привести проект в читаемый вид.

Причин такому “наследству” может быть множество, их мы обсудим ниже. Хочется добавить, что методология у этого проекта была водопад, и в таких случаях часто на рефакторинг выделяется время отдельно по достижению каких-то вех. Видимо, на моё время на проекте оно и выпало.

Что такое рефакторинг?

Начну с базы: что из себя представляет рефакторинг. Рефакторинг - это процесс систематического изменения внутренней структуры программного обеспечения с целью улучшения его понимания, упрощения добавления нового функционала или устранения ошибок. Основная цель рефакторинга – улучшить существующий код без изменения его функциональности. Рефакторинг - это как героизм в IT: мало кто знает об этом, но все пользуются плодами.

В чем ценность рефакторинга для проекта?

Зачем он вообще нужен? Ведь всё и так работает, кнопки жмутся, данные приходят и уходят. Я для себя выделяю три основные причины, по которым рефакторинг необходим:

1. **Улучшение читаемости кода:** Хорошо структурированный, чистый код легче читается и понимается. Это ускоряет процесс разработки, облегчает поддержку кода и ускоряет вхождение в проект нового разработчика.
2. **Упрощение добавления новых функций:** Когда код хорошо организован, добавление новых функций становится проще, менее рискованным и менее трудозатратным для новых сотрудников.
3. **Улучшение производительности:** Иногда рефакторинг может включать оптимизацию кода, что приводит к улучшению производительности.

Как часто нужно проводить рефакторинг?

Если бы меня спросили, я бы сказала - каждый спринт! Но все мы понимаем, что проекты разные, команды разработчиков и методологии разработки разные, и все эти факторы влияют на процесс написания и поддержки кода. Я постараюсь здесь рассмотреть самые популярные варианты.

Частота рефакторинга кода на проекте зависит от ряда факторов, включая размер и сложность проекта, как я писала ранее методологию разработки и культуру команды с их требованиями к качеству кода. Вот некоторые ключевые аспекты, которые помогут определить, как часто следует проводить рефакторинг:

1. **Методология разработки:**

2. **Агильные методологии (например, Scrum):** В этих подходах рефакторинг часто выполняется итеративно и инкрементально. Разработчики могут рефакторить код в рамках каждого спринта или даже чаще. На большинстве проектов, где мы работали по Scrum, задачи на рефакторинг создавались на каждый спринт. Например, при постановке задач на спринт мы учитывали, что они должны быть закрыты за два дня до окончания спринта, а последние два дня выделялись на отладку и рефакторинг.

3. **Традиционные методологии:** В более традиционных моделях, таких как водопад, рефакторинг может быть запланирован как отдельный этап после достижения определённых вех. Мне кажется, такой подход не самым удобным, потому что “определённые вехи” достигаются в какой-то рандомный момент.

4. Размер и сложность проекта:

5. **Маленькие проекты:** Возможно, рефакторинг будет проводиться по мере необходимости, поскольку обзор и изменение кода проще.

6. **Большие и сложные проекты:** Требуется более частый и систематический рефакторинг, чтобы поддерживать код в хорошем состоянии.

7. Частота изменений и добавления функционала:

8. Если проект активно развивается и часто обновляется, рефакторинг может потребоваться чаще для поддержания чистоты кодовой базы и упрощения добавления новых функций.

9. Технический долг:

10. Если в проекте накопился значительный технический долг, частые сеансы рефакторинга могут помочь постепенно улучшить качество кода и снизить сложность.

11. Ресурсы и приоритеты:

12. В зависимости от доступных ресурсов (времени, людей) и приоритетов проекта, рефакторинг может быть более или менее частым, или, как часто бывает, отсутствовать вообще.

Заключение

Важно, чтобы команды и руководители понимали ценность рефакторинга и выделяли на него время и ресурсы. Рефакторинг помогает избежать накопления технического долга и снижает риски, связанные с внедрением новых функций. В конечном итоге, это вклад в качество продукта и удовлетворённость команды разработчиков.

Мне сейчас вспомнилась цитата великого человека, написавшего не менее замечательную книгу "Совершенный код":

Пишите код так, как будто сопровождать его будет склонный к насилию психопат, который знает, где вы живёте. (с) Стив Макконнелл - Совершенный код

Только зарегистрированные пользователи могут участвовать в опросе. [Войдите](#), пожалуйста.

Как часто вы проводите рефакторинг своего кода?

45% Рефакторинг для слабаков

9

35% После отправки в прод

7